Exception Handling – Mobile Number Validation

Observation Task: Week-11 (Experiment No: 11, CO3) Date: __ / __ / 2026

■ Objective / Aim

To write a Java program that validates a mobile number using exception handling concepts: try, catch, throw, throws, finally, multiple catch blocks, built-in exceptions and a user-defined exception.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

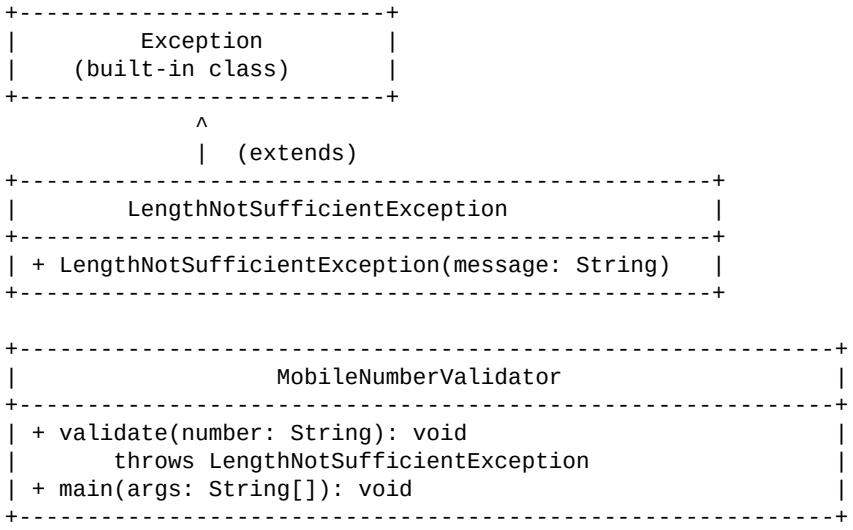
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a user-defined checked exception class named **LengthNotSufficientException** that extends Exception.
2. Create a method **validate(String number)** that declares **throws LengthNotSufficientException**.
3. Inside validate(), check every character of the input:
   - If any character is not a digit (0-9), throw a NumberFormatException.
4. If the length is greater than 10, throw an ArrayIndexOutOfBoundsException.
5. If the length is less than 10, throw a LengthNotSufficientException.
6. Otherwise, print "Valid number".
7. In main(), read the number as a String and call validate() inside a try block.
8. Use multiple catch blocks, one for each of the three exceptions, and print the matching message.
9. Use a finally block to print a completion message.

■ UML Class Diagram



Relationship: LengthNotSufficientException inherits from Exception (it is a checked exception, so the compiler forces it to be caught or declared). MobileNumberValidator uses (association) it by throwing it from validate() and catching it in main(). NumberFormatException and ArrayIndexOutOfBoundsException are built-in unchecked exceptions (subclasses of RuntimeException) that are thrown and caught in the same way.

■ Source Code

```
import java.util.Scanner;

// User-defined exception
class LengthNotSufficientException extends Exception {
    LengthNotSufficientException(String message) {
        super(message);
    }
}

public class MobileNumberValidator {

    // Validates the mobile number
    static void validate(String number)
            throws LengthNotSufficientException {

        // 1. Check for non-digit characters
        for (int i = 0; i < number.length(); i++) {
            char c = number.charAt(i);
            if (c < '0' || c > '9') {
                throw new NumberFormatException(
                    "Number contains non-digit characters");
            }
        }

        // 2. Length greater than 10
        if (number.length() > 10) {
            throw new ArrayIndexOutOfBoundsException(
                "Number exceeds 10 digits");
        }

        // 3. Length less than 10
        if (number.length() < 10) {
            throw new LengthNotSufficientException(
                "Number has fewer than 10 digits");
        }

        System.out.println("Valid number");
    }
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter mobile number: ");
        String number = sc.next();

        try {
            validate(number);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(
                "Invalid Mobile Number-ArrayIndexOutofBounds Exception");
        } catch (LengthNotSufficientException e) {
            System.out.println(
                "Invalid Mobile Number - LengthNotSufficientException");
        } catch (NumberFormatException e) {
            System.out.println(
                "Invalid Mobile Number - NumberFormatException");
        } finally {
            System.out.println("Validation completed.");
            sc.close();
        }
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

`MobileNumberValidator.java`

Compile the program:

```
javac MobileNumberValidator.java
```

Run the compiled program:

```
java MobileNumberValidator
```

■ Output Screen

Run 1

```
Enter mobile number: 9885089465
Valid number
Validation completed.
```

Run 2

```
Enter mobile number: 98567890121
Invalid Mobile Number-ArrayIndexOutofBounds Exception
Validation completed.
```

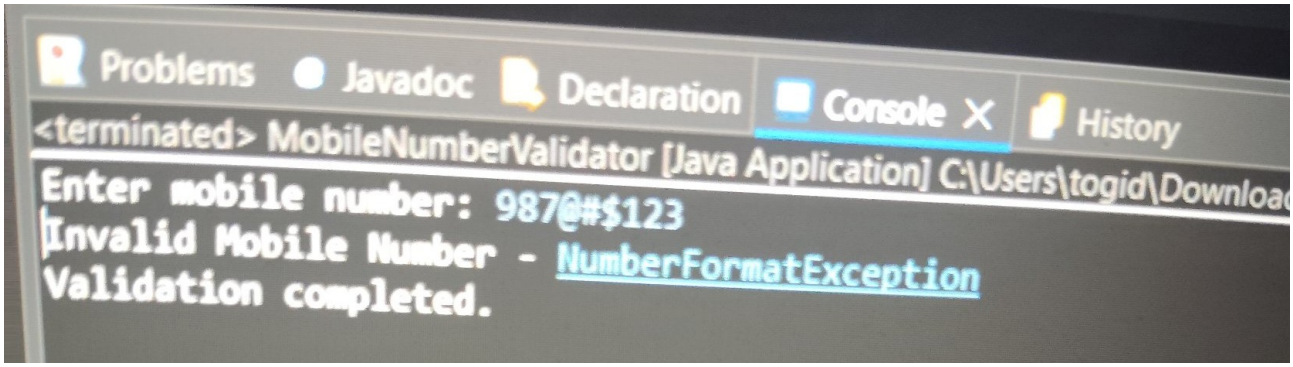
Run 3

```
Enter mobile number: 88664433
Invalid Mobile Number - LengthNotSufficientException
Validation completed.
```

Run 4

```
Enter mobile number: 98ab@123
Invalid Mobile Number - NumberFormatException
Validation completed.
```

Screenshot – actual run in Eclipse (input: 987@#$123)



■ Result

The mobile number validation program was successfully implemented in Java. It correctly identified 9885089465 as valid, and raised ArrayIndexOutOfBoundsException for 11 digits, the user-defined LengthNotSufficientException for 8 digits, and NumberFormatException for non-digit characters.

Note: the non-digit check runs before the length check. This is why 98ab@123 (8 characters) gives NumberFormatException and not LengthNotSufficientException, as the sample output expects.